



ΕΘΝΙΚΟ ΜΕΤΣΟΒΙΟ ΠΟΛΥΤΕΧΝΕΙΟ
ΣΧΟΛΗ ΗΛΕΚΤΡΟΛΟΓΩΝ ΜΗΧΑΝΙΚΩΝ ΚΑΙ ΜΗΧΑΝΙΚΩΝ
ΥΠΟΛΟΓΙΣΤΩΝ

Κάργας Στέφανος Διονύσης Κατσέτης
(el23059) (el23005)

Λειτουργικά Συστήματα Υπολογιστών
6ο εξάμηνο, Ακαδημαϊκή περίοδος 2025-2026

1η Εργαστηριακή Άσκηση:
Κλήσεις συστήματος, διεργασίες και
διαδιεργασιακή επικοινωνία

Αθήνα 2026

Ανάλυση Υλοποίησης `utils.c / utils.h`

Η βιβλιοθήκη αποτελείται από τέσσερις βασικές συναρτήσεις: `writes`, `reads`, `check`, και `sys_print`.

1. Συναρτήσεις `reads` και `writes`:

Οι κλήσεις συστήματος `read()` και `write()` δεν εγγυώνται ότι θα διαβάσουν ή θα γράψουν ολόκληρο το πλήθος των bytes που τους ζητείται με μία μόνο κλήση.

Η λογική και των δύο συναρτήσεων βασίζεται σε βρόχο **do-while**, όπως υποδεικνύεται και στις διαφάνειες του εργαστηρίου. Αυτό διασφαλίζει ότι η συνάρτηση θα επιστρέψει μόνο όταν μεταφερθεί ακριβώς ο αριθμός των bytes που ζητήθηκε.

Χρήση Γενικών Δεικτών (`void*`): Η δήλωση των παραμέτρων ως `const void* msg` και `void* buf` επιτρέπει στο σύστημα να στέλνει μέσω των pipes όχι μόνο αλφαριθμητικά (strings), αλλά και ακατέργαστα δεδομένα (όπως ακέραιους αριθμούς ή δομές), καθαρά byte streams δηλαδή.

Διαχείριση Διακοπών και Σφαλμάτων: Σύμφωνα με τις επισημάνσεις της άσκησης σχετικά με το τι συμβαίνει αν διακοπεί η εκτέλεση ενός system call, η υλοποίησή μας ελέγχει διαρκώς την τιμή επιστροφής των `read()` και `write()`. Αν το σύστημα επιστρέψει τιμή μικρότερη του μηδενός (`rcnt < 0` ή `wcnt < 0`), υποδηλώνεται σφάλμα και η συνάρτηση επιστρέφει -1.

- Η υλοποίησή μας είναι ανθεκτική σε διακοπές από σήματα. Χρησιμοποιώντας την `errno` για την τιμή `EINTR`, η συνάρτηση δεν τερματίζει σε περίπτωση διακοπής, αλλά επαναλαμβάνει την κλήση συστήματος.

2. Συναρτήσεις `check` και `sys_print`:

- `sys_print`: Χρησιμοποιεί την `writes` για να τυπώσει μηνύματα

απευθείας στον file descriptor 1 (οθόνη).

- **check**: Αν δεχθεί αρνητική τιμή (ένδειξη αποτυχίας κάποιου system call), τυπώνει μέσω της writes το διαγνωστικό μήνυμα (*errmsg*) στον file descriptor 2 (stderr) και τερματίζει άμεσα τη διεργασία με την κλήση `_exit(1)`

Οι παραπάνω συναρτήσεις θα χρησιμοποιούνται στα διάφορα μέρη της παρούσας άσκησης, επιδιώκοντας να ελαχιστοποιήσουμε όσο μπορούμε την κλήση συναρτήσεων της Lib C (perror, printf, etc.)

Μέρος 1

Η λογική του κώδικά μας βασίζεται στα εξής σημεία:

1. Διαχείριση Σφαλμάτων με τον File Descriptor 2 (Standard Error):

Το πρόγραμμα αρχικά ελέγχει αν ο χρήστης έδωσε το σωστό πλήθος ορισμάτων (argc/argv). Σε περίπτωση που ο χρήστης κάνει λάθος ή αν κάποιο αρχείο δεν μπορεί να ανοίξει, το πρόγραμμα τυπώνει μηνύματα σφάλματος. Ωστόσο, αντί να χρησιμοποιήσουμε έτοιμες συναρτήσεις όπως η `perror`, χρησιμοποιούμε την **`check()`**. Η ίδια συνάρτηση χρησιμοποιήθηκε και για τον έλεγχο των ορισμάτων γραμμής εντολών περνώντας την τιμή -1.

2. Για το αρχείο εισόδου:

- **`O_RDONLY`**: Ανοίγει το αρχείο αυστηρά και μόνο για ανάγνωση.

Για το αρχείο εξόδου (oflags): Συνδυάσαμε τρεις σημαίες με τον τελεστή bitwise OR (`|`) για να καλύψουμε όλα τα ενδεχόμενα:

- **`O_WRONLY`**: Το αρχείο ανοίγει αποκλειστικά για εγγραφή.
- **`O_CREAT`**: Δίνει εντολή στο σύστημα να δημιουργήσει το αρχείο, σε περίπτωση που ο χρήστης δώσει ένα όνομα αρχείου που δεν υπάρχει.
- **`O_TRUNC`**: Σε περίπτωση που το αρχείο υπάρχει ήδη, η σημαία αυτή φροντίζει να διαγραφούν (να μηδενιστούν) τα παλιά του

περιεχόμενα, ώστε να γραφτεί μόνο το νέο αποτέλεσμα.

Για τα δικαιώματα αρχείου (mode): Επειδή χρησιμοποιήσαμε τη σημαία `O_CREAT` για ενδεχόμενη δημιουργία νέου αρχείου, ήμασταν υποχρεωμένοι να ορίσουμε και τα δικαιώματα πρόσβασης του νέου αρχείου. Γι' αυτό περάσαμε τις σταθερές:

- **S_IRUSR:** Δίνει δικαίωμα ανάγνωσης (read) στον ιδιοκτήτη του αρχείου.
- **S_IWUSR:** Δίνει δικαίωμα εγγραφής (write) στον ιδιοκτήτη του αρχείου

3. snprintf: Καθώς η κλήση `write` δέχεται μόνο συμβολοσειρές (byte streams) και όχι ακεραίους, απαιτείται η μετατροπή του τελικού μετρητή.

Αντί της απλής `sprintf`, προτιμήθηκε αυστηρά η `snprintf(str, sizeof(str), "%d\n", cnt)`. Παρέχοντας το μέγεθος του πίνακα, διασφαλίζεται ότι δεν θα έχουμε overflow στον πίνακα `str`.

Μέρος 2

Ερώτημα 1:

Επισημάνσεις:

- **Ανάκτηση Κωδικού Εξόδου (Exit Status):** Η κλήση `wait(&status)` αποθηκεύει στη μεταβλητή `status` πληροφορίες για τον λόγο τερματισμού της διεργασίας-παιδί.
- **WIFEXITED:** Η `status` είναι κωδικοποιημένη. Χρησιμοποιώντας την `WIFEXITED(status)` επιβεβαιώνουμε ότι το παιδί κάλεσε ομαλά την `_exit()`.
- **wait() :** Η `wait(&status)` περιμένει να πεθάνει οποιοδήποτε παιδί. Εδώ έχουμε μόνο ένα παιδί, άρα η `wait` αρκεί.

Στα επόμενα ερωτήματα με πιο πολλά παιδιά, θα μπορούσαμε

να χρησιμοποιήσουμε την `waitpid()` για να περιμένει ο γονέας αυστηρά το παιδί με το συγκεκριμένο PID.

Ερώτημα 2:

Όταν εκτελεστεί το πρόγραμμα, παρατηρούμε ότι ο γονέας και το παιδί εκτυπώνουν διαφορετικές τιμές για την ίδια μεταβλητή x . Η αλλαγή που έκανε η διεργασία-παιδί στην τιμή της x ($x+=50$) δεν επηρέασε καθόλου την τιμή της x στη διεργασία-γονέα (η οποία έκανε $x+=20$), και αντίστροφα.

Το βασικό συμπέρασμα που επιβεβαιώνεται από την θεωρία είναι ότι η κλήση συστήματος `fork()` δημιουργεί μια νέα διεργασία (το παιδί), η οποία αποκτά ένα ακριβές αντίγραφο του χώρου διευθύνσεων και της μνήμης του γονέα.

Αν και το παιδί κληρονομεί τις μεταβλητές του γονέα (όπως την x με την αρχική της τιμή 100) τη στιγμή ακριβώς που γίνεται η `fork()`, από εκείνο το σημείο και μετά οι δύο διεργασίες είναι εντελώς ανεξάρτητες και διαθέτουν τον δικό τους απομονωμένο χώρο εικονικής και φυσικής μνήμης. Επειδή κάθε διεργασία έχει το δικό της ξεχωριστό αντίγραφο των δεδομένων, οποιαδήποτε τροποποίηση κάνει η μία διεργασία στις μεταβλητές της, δεν είναι ορατή στην άλλη διεργασία.

Ερώτημα 3:

Στο παρόν ερώτημα υλοποιήθηκε η επέκταση του προγράμματος καταμέτρησης χαρακτήρων κάνοντας χρήση της δημιουργίας διεργασιών. Ο στόχος ήταν ο διαχωρισμός των ρόλων: η γονική διεργασία ανέλαβε τη διαχείριση των πόρων (άνοιγμα/κλείσιμο και γράψιμο αρχείων), ενώ η διεργασία-παιδί επιφορτίστηκε με τον υπολογιστικό φόρτο της αναζήτησης:

Η γονική διεργασία καλεί την `fork()` και όπως προβλέπεται από το UNIX, η διεργασία-παιδί κληρονομεί όλους τους ανοιχτούς file descriptors (τους `fdo` και `fdw`). Έτσι, αποκτά άμεση πρόσβαση στα αρχεία δίχως να

χρειαστεί να τα ανοίξει ξανά.

Το παιδί διαβάζει το αρχείο εισόδου μέσω buff_size μεγέθους block και μετράει τις εμφανίσεις (cnt), και επιστρέφει ομαλά κωδικό εξόδου στον γονέα το cnt.

Στο τέλος ο γονέας γράφει το αποτέλεσμα στην stdout.

*Για τη συγκεκριμένη άσκηση, δεν έχουμε δει ακόμα την **pipe** και γενικά δεν ασχολούμαστε με την διεργασιακή επικοινωνία. Πιο καλή πρακτική θα ήταν να επικοινωνήσει ο γονέας με το παιδί μέσω σωλήνωσης και να πάρει το cnt από εκεί. Τώρα το επιστρέφουμε μέσω **_exit(cnt)** που δεν είναι τόσο αποτελεσματικό επειδή ο κωδικός εξόδου δεν χρησιμοποιείται για αυτή τη λειτουργία, αλλά για να ενημερώσει πως τερματίστηκε μια διεργασία. Υποστηρίζει μόνο 8-bit (ως το 255). Αν ο χαρακτήρας βρεθεί > 255 φορές, ο γονέας θα λάβει λάθος τιμή και θα γράψει λάθος νούμερο στο αρχείο!*

Ερώτημα 4:

Η υλοποίηση βασίστηκε στη χρήση της κλήσης συστήματος execve():

Αντικατάσταση Μνήμης στο Παιδί (execve) : Στο μπλοκ εκτέλεσης της διεργασίας-παιδί, το παιδί προετοιμάζεται για την κλήση της execve().

Επιλέξαμε την execve() έναντι άλλων πιθανών αφού αυτή είναι πραγματικά μόνο κλήση συστήματος.

Δημιουργήσαμε έναν πίνακα δεικτών χαρακτήρων (char *param[]), ο οποίος περιέχει τα ορίσματα που πρέπει να περαστούν στο νέο πρόγραμμα: το όνομα του εκτελέσιμου προγράμματος ("a1.1-C") ακολουθούμενο από το αρχείο εισόδου, το αρχείο εξόδου, τον χαρακτήρα, και τερματίζεται υποχρεωτικά με NULL.

Όταν καλείται η execve(), το λειτουργικό σύστημα καταστρέφει την τρέχουσα εικόνα μνήμης της διεργασίας και φορτώνει στη θέση της το εκτελέσιμο αρχείο "../meros1/char-count-with_c_lib/a1.1-C".

Αν η κλήση αυτή πετύχει, η execve δεν επιστρέφει ποτέ, καθώς ο παλιός

κώδικας έχει πλέον αντικατασταθεί. Οι εντολές διαχείρισης σφάλματος (τύπωμα λάθους και `_exit(1)`) που βρίσκονται ακριβώς κάτω από την `execve()` θα εκτελεστούν αποκλειστικά και μόνο στην περίπτωση που η φόρτωση του νέου προγράμματος αποτύχει (π.χ. αν δεν βρεθεί η διαδρομή του εκτελέσιμου).

Μέρος 3

Χρήση και Διαχείριση Σημάτων (Signals):

- **SIGCHLD**: Όταν ένα παιδί τερματίζει, το Λειτουργικό Σύστημα στέλνει αυτόματα το σήμα SIGCHLD στη γονική διεργασία.

Η γονική διεργασία το πιάνει μέσω της `sigchld_handler`, η οποία έχει υλοποιηθεί με την `sigaction` (πιο ασφαλής για χειρισμό σημάτων από τη `signal` που είναι αναξιόπιστη) και `flag` το **SA_RESTART** (ώστε να μην διακόπτονται βίαια κλήσεις συστήματος).

Μέσα στον handler για το SIGCHLD, καλούμε την **waitpid με WNOHANG** μέσα σε έναν βρόχο `while`. Με το `flag WNOHANG`, αν το παιδί δεν έχει τερματίσει ακόμα, η `waitpid()` δεν θα περιμένει. Θα επιστρέψει με τιμή 0, επιτρέποντας στον γονέα να συνεχίσει την κανονική του εκτέλεση.

Χρησιμοποιούμε `while` διότι στο UNIX τα σήματα δεν μπαίνουν σε ουρά. Αν πολλά παιδιά τερματίσουν ταυτόχρονα, ο γονέας μπορεί να λάβει μόνο ένα SIGCHLD. Έτσι εξασφαλίζουμε ότι ο γονέας θα συλλέξει όλα τα παιδιά που έχουν τερματίσει εκείνη τη στιγμή, μειώνοντας τη μεταβλητή `active_children` όσο πρέπει.

- Διαχείριση **SIGINT (Ctrl+C)** : Τα παιδιά το αγνοούν πλήρως (`signal(SIGINT, SIG_IGN)`).

Για τον γονέα, ο *sigint_handler*, αλλάζει ακαριαία την τιμή μιας global flag που ορίζουμε, *sigint_flag* = -1 (τύπου volatile sig_atomic_t). Η πραγματική εκτύπωση των ενεργών παιδιών γίνεται με απόλυτη ασφάλεια στη ροή της main.

→ Απαγορεύεται αυστηρά να καλούμε μέσα σε συνάρτηση χειρισμού συναρτήσεις που χρησιμοποιούν μηχανισμούς κλειδώματος όπως η *snprintf*, *sprintf*, *printf* διότι οδηγούν σε *deadlocks* και κολλούν το πρόγραμμα.

→ Μία άλλη λύση θα ήταν να υλοποιούσαμε εξωτερικά μια χειροκίνητη συνάρτηση *casting* από *int* σε *char*, την οποία θα καλεί η συνάρτηση χειρισμού μας *sigint_handler*

Διαμοιρασμός Φόρτου: Ο γονέας διαβάσει το συνολικό μέγεθος του input αρχείου με την *stat()* (την επιλέξαμε έναντι της *fstat()* αφού ο γονέας δεν θα ανοίξει το input file οπότε θα αποφύγει περιττές κλήσεις συστήματος).

Επίσης, υπολογίζει το μέγεθος του κομματιού (*chunk* = *size_r* / *P*), και κάθε παιδί κάνει το δικό του ανεξάρτητο *open()*.

Στη συνέχεια, χρησιμοποιεί την *lseek()* για να μεταβεί στο αντίστοιχο *offset*, διαβάζοντας αυστηρά μέχρι το όριο (*end*) που του έχει ανατεθεί.

Pipes: Για την επιστροφή των μερικών αθροισμάτων (*cnt*), τα παιδιά γράφουν κατευθείαν *byte streams* (έναν *int* που είναι 4 bytes) μέσα στο *pipe*.

Αντί να χρησιμοποιηθούν οι *read/write* του συστήματος, χρησιμοποιούνται οι *reads()* και *writes()*. Αυτές περικλείουν τις κλήσεις συστήματος σε βρόχους *while*, εξασφαλίζοντας ότι δεν θα υπάρξουν ποτέ ημιτελείς αναγνώσεις/εγγραφές.

Event Loop: Ο γονέας εισέρχεται στον βρόχο `while(active_children > 0)`.

Πριν από το βρόχο, μπλοκάρουμε τα σήματα SIGINT και SIGCHLD με την βοήθεια της ***sigprocmask()***. Μέσα στο loop, ελέγχουμε τη σημαία *sigint_flag* και, αν είναι ενεργή, τυπώνουμε το μήνυμα.

Στη συνέχεια, χρησιμοποιούμε την ***sigsuspend()*** αντί για την `pause()`. Ο λόγος που το επιλέξαμε αυτό είναι διότι η `sigsuspend` ξεμπλοκάρει τα σήματα και θέτει τη διεργασία σε αναμονή σε μία ατομική κίνηση (δηλαδή ακαριαία).

Έτσι, εκμηδενίζεται η πιθανότητα να έρθει ένα σήμα και να χαθεί ανάμεσα στον έλεγχο της σημαίας και στον ύπνο της διεργασίας, κάτι που θα προκαλούσε πρόβλημα Race Condition με την `pause()`.

Μέρος 4

FRONT-END:

1. Δημιουργία IPC: Καθώς ο Front-end και ο Dispatcher αποτελούν ξεχωριστές διεργασίες με απομονωμένους χώρους μνήμης, η μεταξύ τους επικοινωνία μπορεί να υλοποιηθεί μέσω (ανώνυμων) σωληνώσεων, αφού θα έχουν σχέση γονέα-παιδιού αντίστοιχα.

Επειδή οι σωληνώσεις αυτές επιτρέπουν μόνο μονόδρομη μεταφορά δεδομένων, δημιουργούμε με την κλήση `pipe()` δύο ξεχωριστές σωληνώσεις:

- **ftod** (Front-To-Dispatcher): Για την αποστολή εντολών από τον χρήστη προς τον Dispatcher.
- **dtof** (Dispatcher-To-Front): Για τη λήψη των απαντήσεων και της κατάστασης από τον Dispatcher.

2. Δημιουργία Διεργασιών: Η δημιουργία του Dispatcher πραγματοποιείται με την `fork()`. Αμέσως μετά τη δημιουργία της διεργασίας-παιδιού:

Η διεργασία-παιδί (Dispatcher) κλείνει τα άκρα των σωληνώσεων

που δεν της χρειάζονται (ftod και dtod), ενώ ο γονέας (Front-end) κλείνει τα αντίθετα άκρα (ftod και dtod) που δεν του χρειάζεται, όπως προβλέπεται στο μοντέλο του UNIX.

Η διεργασία-παιδί αντικαθιστά τον εαυτό της με το εκτελέσιμο του Dispatcher μέσω της κλήσης συστήματος *execve()*, η οποία δέχεται μόνο strings ως παραμέτρους στη λίστα παραμέτρων *param*.

Δεδομένου ότι η *execve* ξεκινά ένα νέο πρόγραμμα, οι ακέραιες μεταβλητές των file descriptors των pipes χάνονται. Έτσι, οι περιγραφητές μετατρέπονται σε συμβολοσειρές μέσω της *snprintf* και περνούν στον Dispatcher ως ορίσματα γραμμής εντολών.

3. Αλληλεπίδραση και Μορφοποίηση Δεδομένων :

- **Πρωτόκολλο Μηνυμάτων μέσω Pipes :** Επιλέξαμε την αποστολή byte streams σταθερού μεγέθους 64-bytes.

Το μήνυμα προς τον Dispatcher (*out_b*) αρχικοποιείται με χαρακτήρες κενού (*memset(out_b, ' ', 64)*) και στον πρώτο χαρακτήρα τοποθετείται η εντολή που έχει δώσει χρήστη (+, -). Για το command i, το front-end στέλνει ένα σήμα στον dispatcher μέσω του **SIGUSR1**, αναγκάζοντάς τον να παρατήρει ότι κάνει, να υπολογίσει την πρόοδο εκείνη τη στιγμή, και να τη γράψει στο pipe επιστροφής.

Μετά το front-end περιμένει την απάντηση του και τυπώνει το αποτέλεσμα (**Handshake**).

Η χρήση των *custom writes()* και *reads()* της *utils.c* εγγυάται ότι τα δεδομένα δεν θα κοπούν και το byte stream θα διαβαστεί πλήρως από την άλλη πλευρά.

- **Handshake:** Η απάντηση του Dispatcher γίνεται μέσω της *sscanf*, μέσω της οποίας (χρησιμοποιώντας συγκεκριμένους μορφοποιητές που επιλέξαμε για την μεταφορά σταθερού μήκους ακέραιων δεδομένων *%3d%10d%2d*) εξάγουμε το ποσοστό προόδου, τους χαρακτήρες που βρέθηκαν και τους ενεργούς workers.

4. Τερματισμός: Όταν η διαδικασία αναζήτησης φτάσει στο 100% ή όταν ο χρήστης δώσει την εντολή εξόδου (q). Στέλνει το σήμα SIGTERM (το οποίο μπορεί να γίνει handle σε αντίθεση με το SIGKILL και αφήνει περιθώρια στον Dispatcher να τελειώσει ομαλά τις ενεργές διεργασίες του) στη διεργασία του Dispatcher μέσω της kill(), ζητώντας τον τερματισμό της.

Στη συνέχεια, καλεί ρητά την waitpid(d_pid, NULL, 0), θέτοντας τον εαυτό του σε αναμονή μέχρι να επιβεβαιωθεί ο θάνατος του παιδιού.

Για να παραμείνει το Front-end interactive και να μην μπλοκάρει περιμένοντας απαντήσεις από Dispatcher ή τον Χρήστη, χρησιμοποιήθηκε η κλήση συστήματος **poll()**. Αυτή επιτρέπει στη διεργασία να παρακολουθεί ταυτόχρονα δύο πηγές εισόδου: το πληκτρολόγιο του χρήστη (stdin) και το pipe από τον Dispatcher.

Θα μπορούσε να χρησιμοποιηθεί και η select() για αυτό τον σκοπό.

DISP_TYPES.H:

Το header αρχείο disp_types.h λειτουργεί ως διεπαφή ανάμεσα στον Dispatcher και τους Workers, ορίζοντας τις δομές δεδομένων που είναι απαραίτητες για τον συντονισμό της εργασίας.

1. Struct Task

Η struct Task αποτελεί τη βασική μονάδα πληροφορίας που στέλνεται από τον Dispatcher προς τον Worker μέσω των pipes.

- **start & size:** Ορίζουν το κομμάτι του αρχείου πρέπει να διαβάσει ο εργάτης.

- quit: Flag που επιτρέπει τον ομαλό και ασφαλές τερματισμό του Worker. Αντί ο Dispatcher να σκοτώνει βίαια τους εργάτες, τους στέλνει ένα Task με quit=1, επιτρέποντάς τους να κλείσουν τα αρχεία τους και να τερματίσουν ομαλά.

Ουσιαστικά το Task αποτελεί το καθήκον και πληροφορίες για αυτό, που θα αναλάβει ο Worker από τον Dispatcher.

2. Struct Worker

Αυτή η δομή χρησιμοποιείται αποκλειστικά από τον Dispatcher.

- pid: Απαραίτητο για να ξέρει ο Dispatcher τα παιδιά που έχει δημιουργήσει.
- fd_send & fd_receive: Αποθηκεύουμε τους περιγραφητές των σωληνώσεων, για κάθε εργάτη ξεχωριστά, προσφέροντας στον Dispatcher πληροφορία για το πως θα μπορέσει να επικοινωνήσει με κάθε ενεργό εργάτη.
- is_active: Μας επιτρέπει να γνωρίζουμε αν μια θέση στον πίνακα εργατών είναι κατειλημμένη ή αν μπορούμε να κάνουμε spawn έναν νέο εργάτη εκεί (δηλαδή δηλώνει αν έχουμε ενεργό εργάτη εκεί).

3. Struct JobInfo

Συγκεντρώνει τα καθολικά δεδομένα της αναζήτησης (όνομα αρχείου, χαρακτήρας-στόχος, συνολικό μέγεθος).

4. Πρωτόκολλο Σταθερού Μήκους (MSG_SIZE)

Ο ορισμός `#define MSG_SIZE 64` διασφαλίζει ότι η επικοινωνία μεταξύ Front-end και Dispatcher ακολουθεί ένα αυστηρό πρωτόκολλο σταθερού μήκους, όπως προΐδεάσαμε ότι θα κάνουμε στην περιγραφή της υλοποίησης του Dispatcher.

DISPATCHER:

Ο Dispatcher είναι ο ενδιάμεσος μεταξύ του Front-end και των Workers.

1. Διανομή Εργασίας και Work Pool

Ο Dispatcher τεμαχίζει σε μικρά chunks μεγέθους CHUNK (1024 bytes).

Μέσα στο loop, ο Dispatcher ελέγχει τον πίνακα εργατών. Αν βρει κάποιον που είναι ενεργός αλλά όχι απασχολημένος (*!worker_busy*), του αναθέτει το επόμενο διαθέσιμο chunk.

2. Πολυπλεξία I/O

Επειδή ο Dispatcher πρέπει να προσέχει/ακούει ταυτόχρονα πολλές πηγές, χρησιμοποιήθηκε η poll():

- Παρακολουθεί το pipe από το Front-end για εντολές προσθήκης (+) ή αφαίρεσης (-) εργατών (*ftod[0]*).
- Παρακολουθεί δυναμικά όλα τα pipes επιστροφής από τους ενεργούς Workers, που θα του επιστρέφουν τα αποτελέσματα
- Χρησιμοποιούμε τον βοηθητικό πίνακα *poll_to_workers* για να αντιστοιχίσουμε το file descriptor που έχει έτοιμα δεδομένα προς παράδοση με τον σωστό worker στον πίνακα *workers*.

3. Ανθεκτικότητα σε Σφάλματα

Ο Dispatcher είναι σχεδιασμένος να μην χάνει δεδομένα αν ένας worker τερματιστεί βίαια:

- Μέσω του *sigchld_handler* και της *waitpid()*, ο Dispatcher εντοπίζει ποιο παιδί πέθανε.
- Αν ο worker που πέθανε ήταν απασχολημένος (*worker_busy = 1*), ο Dispatcher εκτελεί την εντολή *current -= current_task[i].size*. Με αυτόν τον τρόπο, επιστρέφει το χαμένο chunk πίσω στο pool, ώστε

να ανατεθεί στον επόμενο διαθέσιμο worker.

4. Διαχείριση Σημάτων

- SIGUSR1 (Info Request): Όταν ληφθεί αυτό το σήμα από το Front-end, ο Dispatcher υπολογίζει το ποσοστό προόδου βάσει των bytes που έχουν ήδη επεξεργαστεί (*total_processed*) και στέλνει ένα μορφοποιημένο μήνυμα 64-bytes πίσω στο Front-end με χρήσιμες πληροφορίες για αναφορά που επιθυμεί να παρακολουθεί ο χρήστης.
- SIGCHLD: Χρησιμοποιείται για τον άμεσο καθαρισμό των zombie διεργασιών και την αυτόματη αντικατάσταση (*spawn*) του εργάτη που χάθηκε, διατηρώντας **τον αριθμό των workers σταθερό σύμφωνα με τις επιθυμίες του χρήστη.**

5. Αυξομείωση Εργατών

- Στο +, καλεί την *spawn()* για να δημιουργήσει νέα pipes και νέα διεργασία.
- Στο -, στέλνει ένα Task με τη σημαία *quit=1*.

Αν ο εργάτης ήταν *busy*, η εργασία του ακυρώνεται και επιστρέφει στο pool για να την πάρει κάποιος άλλος.

WORKER:

Ο Worker αποτελεί το back-end της εφαρμογής. Εκτελεί αυστηρά τις εντολές που δέχεται από τον Dispatcher.

Ο Worker εισέρχεται σε έναν ατέρμονα βρόχο αναμονής, χρησιμοποιώντας τη συνάρτηση *reads()*.

- Κάθε ανάγνωση αντιστοιχεί σε μία δομή Task. Ο Worker παραμένει

μπλοκαρισμένος μέχρι ο Dispatcher να του στείλει δουλειά.

- Ο Worker τερματίζει ομαλά μόνο αν η *reads()* επιστρέψει 0 (κλείσιμο της σωλήνωσης από τον Dispatcher) ή αν το πεδίο *quit* της δομής Task είναι ίσο με

Για να διασφαλιστεί ότι η εφαρμογή δεν θα καταρρεύσει από τυχαία σήματα του χρήστη, εφαρμόζουμε την εξής πολιτική:

Αγνόηση Σημάτων: Μέσω της *signal()*, ο Worker αγνοεί κάποια σήματα τερματισμού που μπορεί να δημιουργηθούν από τον χρήστη. Αν δεν αγνοούσε το σήμα, θα τερμάτιζε πρόωρα. Δέχεται μόνο εντολές από τον Dispatcher.

Εκτέλεση Αναζήτησης και Διαχείριση Αρχείου

- **Ανεξάρτητοι Περιγραφητές:** Κάθε Worker εκτελεί το δικό του *open()* στο αρχείο εισόδου. Αυτό επιτρέπει σε κάθε εργάτη να έχει τον δικό του δείκτη θέσης, αποφεύγοντας τις συγκρούσεις που θα προέκυπταν αν μοιράζονταν έναν κοινό file descriptor.
- Με τη χρήση της *lseek()*, ο Worker πηδά ακαριαία στο σημείο έναρξης του Task που του ανατέθηκε. Διαβάζει το αρχείο σε blocks μεγέθους *BUFF_SIZE*.
- Μόλις ολοκληρωθεί η μέτρηση του συγκεκριμένου chunk, το αποτέλεσμα στέλνεται ως ένας απλός ακέραιος (ακατέργαστο byte stream) πίσω στον Dispatcher μέσω της *writes()*.

Επίσης, χρησιμοποιούμε τη *usleep()* ~ ανά διάβασμα ενός block από το αρχείο. Αυτή η σκόπιμη καθυστέρηση επιτρέπει:

- Την εποπτεία της παράλληλης εκτέλεσης σε πραγματικό χρόνο μέσω του Front-end.
- Την επιβεβαίωση ότι η δυναμική προσθήκη/αφαίρεση εργατών λειτουργεί σωστά ενώ η αναζήτηση βρίσκεται σε εξέλιξη.